

Feeding Frenzy 2: unpacking a self-checksumming fish tank

PopCap's *Feeding Frenzy 2* (2006) keeps every sprite, sound, and cutscene in one ~14 MB file: `FF2.saf`. No loose assets, just an opaque blob. Here is how it comes apart, and the single trick that makes putting it back together interesting.

The header is trivial:

```
struct saf_header { // little-endian
    char magic[4]; // "FFAS"
    u32 version; // 1
    u32 toc_offset; // jump here for index
};
```

Everything from `0x0C` to `toc_offset` is the raw concatenation of every packed file. The table of contents sits at the end:

```
struct toc {
    u32 version;
    u8 global_checksum[16];
    u32 entry_count;
    struct {
        u32 data_start, data_size;
        u8 checksum[16];
        u16 path_len;
        char path[path_len]; // NUL-term
    } entries[entry_count];
};
```

Walk the entries, `seek(data_start)`, read `data_size`, write to `path`. Done: 1940 files fall out — 1239 PNG + 246 JPG sprites, 363 XML layouts, 54 OGG tracks, and 29 Theora videos for the cutscenes. A Kaitai Struct `.ksy` pins the whole format down in about 50 lines.

Extraction is free; *repacking* is where the game bites back. Every file carries a 16-byte checksum, and the TOC carries a global one computed over `entry_count` followed by each entry's (`start`, `size`, `checksum`, `path_len`, `path`). On load, the engine recomputes both and compares. Feed it a stock `md5()` and the result *looks* right — correct 16 bytes — yet the game rejects the archive.

The checksum is MD5, RFC 1321 to the letter: the same 64 sine constants, the same rotation schedule, the same padding. Only the four initialization words have been swapped:

```
// stock MD5 Feeding Frenzy 2
A = 0x67452301 -> 0xfb132803
B = 0xefcdab89 -> 0x06e828f3
C = 0x98badcfe -> 0xa12a051c
D = 0x10325476 -> 0x9bedf4fc
```

Four magic numbers, lifted straight out of `ff2.exe` in Binary Ninja. Drop them into any MD5 implementation and the digests suddenly match the ones stored in the original archive. That is the entire “DRM”: a hash that is 99% standard and 1% lie, betting you will reach for `hashlib` and never notice the initial state is off.

With the real IV, a ~100-line repacker rebuilds a byte-identical archive from the extracted `resource/` tree — recomputing every per-file digest and the global one — and the game loads it without complaint. Swap a sprite, retranslate the XML, drop in your own fish: the tank is open.

Extractor, repacker, and format notes:

<https://github.com/xusheng6/feeding-frenzy-saf>